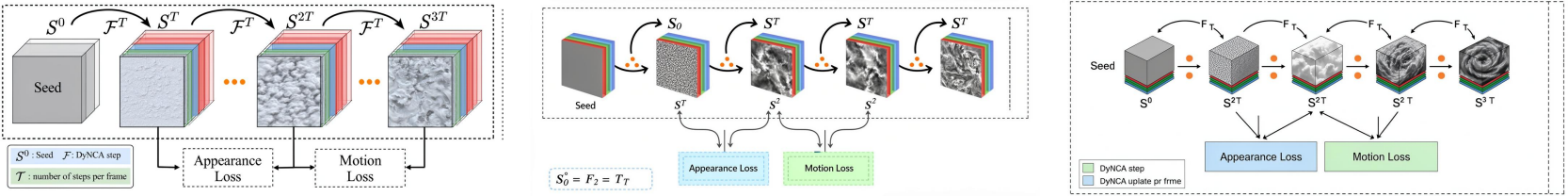
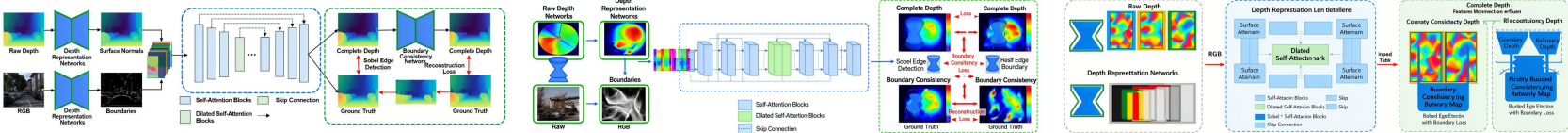


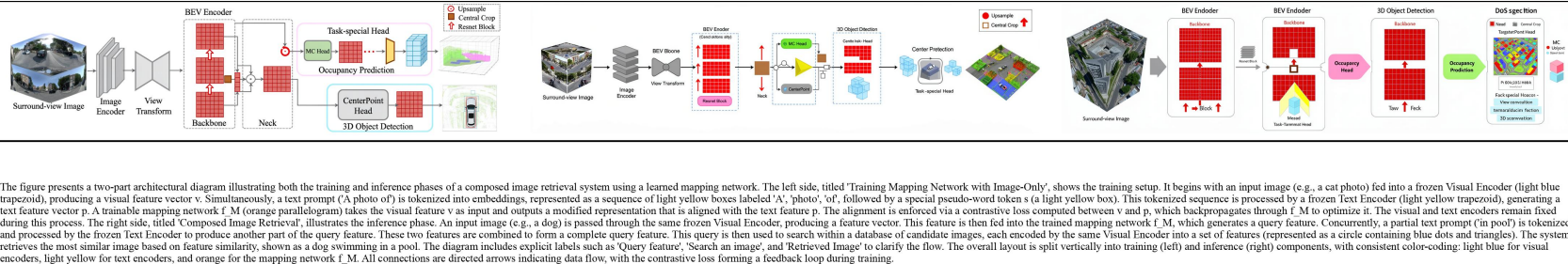
The figure presents an overview of the DyNCA (Dynamic Neural Cellular Automata) framework, illustrating its iterative process for generating dynamic texture videos. The global layout is a horizontal pipeline showing a sequence of state transformations, starting from an initial seed and progressing through multiple steps to produce a series of frames. The structure is linear and sequential, with four main state blocks aligned horizontally: S^0 , S^1 , S^2 , and S^3 , each represented as a 3D cube with multiple layers, symbolizing a multi-channel state tensor. These cubes are connected by curved black arrows labeled F , indicating the application of T steps of the DyNCA update function F per frame. The first cube, S^0 , is labeled 'Seed' and is filled with solid gray, representing the initial state. Subsequent cubes show increasingly complex textures: S^1 displays a fine-grained, bumpy surface; S^2 shows more pronounced cloud-like structures; and S^3 exhibits even more detailed, turbulent patterns, suggesting progressive refinement over iterations. Each cube has colored side faces (red, green, blue) to indicate channel dimensions, and the front face contains the evolving texture. Between the cubes, orange dots represent intermediate states not shown explicitly. Below the sequence, two dashed rectangular boxes labeled 'Appearance Loss' and 'Motion Loss' receive inputs from S^0 , S^1 , and S^2 via downward arrows, indicating that these losses are computed based on the generated frames to guide training. The Appearance loss compares the generated frames against a target appearance, while the motion loss evaluates temporal consistency or motion dynamics. A legend at the bottom left defines the notation: S^0 as the seed, F as the DyNCA step, and T as the number of steps per frame. A dashed vertical line at the far right suggests the continuation of the sequence beyond S^3 . The entire diagram is enclosed within a dashed horizontal boundary, emphasizing the training phase where losses are computed. The visual design uses clean lines, distinct colors for losses (light blue for appearance, light green for motion), and clear labeling to convey the iterative generation and loss computation process central to DyNCA's training.



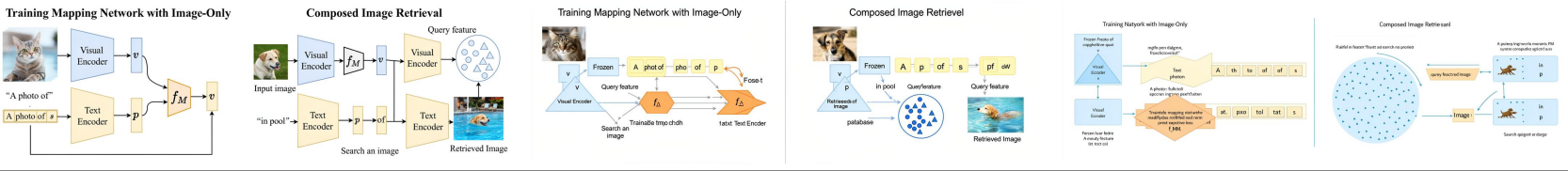
The figure presents an end-to-end deep learning architecture for depth completion, structured into three main stages: input representation, feature processing with self-attention, and output refinement with boundary consistency. The global layout is left-to-right, starting with raw inputs, progressing through intermediate representations, and culminating in a refined depth output with associated losses. On the far left, two inputs are shown: 'Raw Depth' (a color-coded depth map) and 'RGB' (a real-world scene image). These inputs are fed into two parallel 'Depth Representation Networks', each depicted as a blue hourglass-shaped encoder-decoder structure with green borders. The top network processes 'Raw Depth' to predict surface normals, visualized as a colorful segmented image. The bottom network processes 'RGB' to predict boundaries, shown as a grayscale edge map. Both predicted outputs are then concatenated into a multi-channel feature tensor, represented by stacked colored rectangles, which serves as input to the next stage. The central part of the diagram, enclosed in a blue dashed box, represents the core self-attention network. This module consists of a sequence of rectangular blocks symbolizing convolutional layers or attention heads. A light green block in the center denotes a dilated self-attention block, while surrounding light blue blocks represent standard self-attention blocks. Dotted arrows indicate skip connections between layers, forming a residual-like structure. Below this module, a legend identifies the components: a light blue rectangle is labeled 'Self-Attention Blocks', a light green rectangle is labeled 'Dilated Self-Attention Blocks', and a dotted arrow signifies 'Skip Connection'. The rightmost section, enclosed in a green dashed box, details the output refinement and loss computation. The self-attention network outputs a 'Complete Depth' map (color-coded), which is fed into a 'Boundary Consistency Network' (another blue hourglass structure) to produce a refined boundary map. Simultaneously, the 'Complete Depth' map undergoes 'Sobel Edge Detection' to extract its edges. The resulting boundary maps from both paths are compared against the ground truth boundary (derived from the original 'Ground Truth' depth map) using a 'Boundary Consistency Loss', indicated by a red double-headed arrow. Additionally, the 'Complete Depth' map is compared directly to the 'Ground Truth' depth map using a 'Reconstruction Loss', shown by another red double-headed arrow. These losses guide the training of the entire network to produce structured and accurate depth predictions.



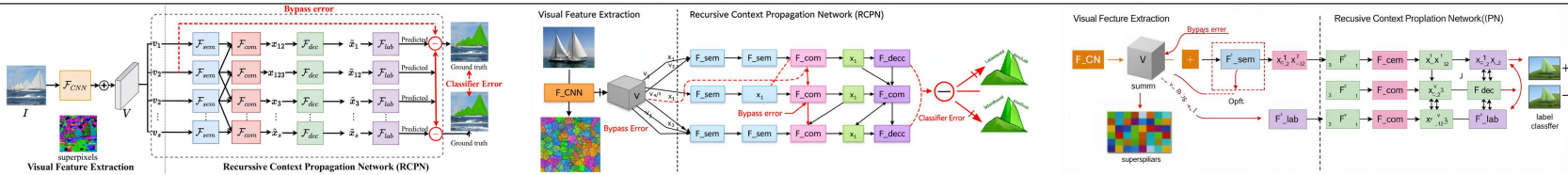
The figure presents the architectural pipeline of the proposed UltimateDO system, designed for multi-task perception from surround-view images. The global layout is a left-to-right data flow, starting with input images and progressing through encoding, view transformation, feature extraction, and task-specific heads, culminating in two output visualizations: occupancy prediction and 3D object detection. The process begins on the far left with a 'Surround-view Image' depicted as a 360-degree panoramic cube, showing real-world street scenes. This input feeds into an 'Image Encoder', represented by three stacked gray rectangular blocks, symbolizing a deep neural network (likely a CNN backbone). The encoded features then pass through a 'View Transform' module, shown as a gray bowtie-shaped block, which projects the image features into a bird's-eye view (BEV) representation. The output of this stage is a grid of red square tiles, indicating BEV feature maps. These BEV features enter the 'BEV Encoder' section, divided into 'Backbone' and 'Neck'. The Backbone consists of three stacked red grid tiles connected vertically by upward-pointing red arrows labeled 'Resnet Block', indicating progressive feature extraction with residual connections. The Neck includes a central crop operation (marked by a brown square icon) applied to one of the feature levels, followed by a concatenation step (indicated by a small 'C' in a circle). The final output of the BEV Encoder is a 'Task-special Head' (highlighted in pink), which branches into two task-specific heads: 'Occupancy Prediction' and '3D Object Detection'. The 'Occupancy Prediction' head is a green rounded rectangle, followed by a sequence of red grid tiles, then a yellow trapezoidal block representing a temporal fusion module (optional, activated by a red switch as per caption), and finally a stack of light blue cubes symbolizing a 3D convolutional or upsampling layer. The output is a 3D occupancy grid visualization, showing colored regions representing different semantic classes (e.g., road, vegetation, vehicles). The '3D Object Detection' head is a green rounded rectangle, followed by a sequence of red grid tiles, then a yellow trapezoidal block representing a temporal fusion module (optional, activated by a red switch as per caption), and finally a stack of light blue cubes symbolizing a 3D convolutional or upsampling layer. The output is a 3D occupancy grid visualization, showing colored regions representing different semantic classes (e.g., road, vegetation, vehicles). The second branch leads to the '3D Object Detection' head, which is a green rounded rectangle, followed by a sequence of red grid tiles, then a yellow trapezoidal block representing a temporal fusion module (optional, activated by a red switch as per caption), and finally a stack of light blue cubes symbolizing a 3D convolutional or upsampling layer. The output is a 3D occupancy grid visualization, showing colored regions representing different semantic classes (e.g., road, vegetation, vehicles). The legend in the upper right corner defines the icons: a red circle with a dot for 'Upsample', a brown square for 'Central Crop', and a red upward arrow for 'Resnet Block'. All other components follow conventions from Figure 1-ref (fig.1). The dashed boxes indicate modular components that can be replaced, and the light blue region (temporal fusion) is optional. The entire diagram emphasizes modularity, multi-task learning, and BEV-based perception.



The figure presents a two-part architectural diagram illustrating both the training and inference phases of a composed image retrieval system using a learned mapping network. The left side, titled 'Training Mapping Network with Image-Only', shows the training setup. It begins with an input image (e.g., a cat photo) fed into a frozen Visual Encoder (light blue trapezoid), producing a visual feature vector v . Simultaneously, a text prompt (A photo of) is tokenized into embeddings, represented as a sequence of light yellow boxes labeled 'A', 'photo', 'of', followed by a special pseudo-word token s (a light yellow box). This tokenized sequence is processed by a frozen Text Encoder (light yellow trapezoid), generating a text feature vector p . A trainable mapping network f_M (orange parallelogram) takes the visual feature v as input and outputs a modified representation that is aligned with the text feature p . The alignment is enforced by a contrastive loss computed between v and p , which backpropagates through f_M to optimize it. The visual and text encoders remain fixed during this process. The right side, titled 'Composed Image Retrieval', illustrates the inference phase. An input image (e.g., a dog) is passed through the same frozen Visual Encoder, producing a feature vector. This feature is then fed into the trained mapping network f_M , which generates a query feature. Concurrently, a partial text prompt ('in pool') is tokenized and processed by the frozen Text Encoder to produce another part of the query feature. These two features are combined to form a complete query feature. This query is then used to search within a database of candidate images, each encoded by the same Visual Encoder into a set of features (represented as a circle containing blue dots and triangles). The system retrieves the most similar image based on feature similarity, showing a dog swimming in a pool. The diagram includes explicit labels such as 'Query feature', 'Search an image', and 'Retrieved Image' to clarify the flow. The overall layout is split vertically into training (left) and inference (right) components, with consistent color-coding: light blue for visual encoders, light yellow for text encoders, and orange for the mapping network f_M . All connections are directed arrows indicating data flow, with the contrastive loss forming a feedback loop during training.



The figure presents a complete flow diagram of the Recursive Context Propagation Network (RCPN) for semantic segmentation, structured into two main stages: Visual Feature Extraction and Recursive Context Propagation Network (RCPN), separated by a dashed vertical boundary. [1] Global Layout and Structure: The diagram is horizontally organized from left to right, depicting the data flow through the network. The left section, labeled 'Visual Feature Extraction', processes an input image I (a sailboat scene) through an orange rectangular module labeled F_{CNN} , representing a convolutional neural network. Its output is combined with a gray 3D block labeled V (visual feature map) via a summation operator $(+)$. Below this, a colorful segmented image labeled 'superpixels' indicates region-based processing. From this stage, multiple visual feature streams $v_1, v_2, \dots, v_s$ are extracted and fed into the RCPN. In the RCPN stage, each v_i enters a light blue rectangular module labeled F_{sem} (semantic feature extractor). Outputs $x_1, x_2, \dots, x_s$ from these modules feed into a recursive structure. Each x_i connects to a pink rectangular module labeled F_{com} (context combiner), which merges features across levels. For example, x_1 feeds into F_{com} to produce x_{12} ; x_2 also feeds into F_{com} to produce x_{12} ; and x_3 feeds into another F_{com} to produce x_{123} , illustrating recursive combination. These combined features then pass through light green rectangular modules labeled F_{dec} (decoder), generating reconstructed outputs $\hat{x}_1, \hat{x}_{12}, \hat{x}_{123}, \dots, \hat{x}_s$. Each reconstructed output is processed by a purple rectangular module labeled F_{lab} (label classifier) to produce a predicted segmentation. A red dashed arrow labeled 'Bypass error' runs from $\hat{x}_1$ back to x_1 , indicating a feedback mechanism for error propagation. On the far right, the predicted segmentations are compared with ground truth images (two versions shown: one with green hill, one with green sail) via a red circular subtraction operator $(-)$, producing a 'Classifier Error' signal, highlighted in red text and arrows. [2] Connections and Arrows: Solid black arrows indicate forward propagation: from $I \rightarrow F_{CNN} \rightarrow V \rightarrow F_{sem} \rightarrow x_1 \rightarrow F_{com} \rightarrow x_{12} \rightarrow F_{dec} \rightarrow \hat{x}_{12} \rightarrow F_{lab} \rightarrow \text{Predicted}$. Red solid arrows connect the predicted outputs and ground truth to the subtraction node, which outputs the 'Classifier Error'. This error signal is visually emphasized with bold red lines and text, indicating its role in training or refinement. [3] Bypass Error and Feedback: A red dashed arrow labeled 'Bypass error' loops from $\hat{x}_1$ back to x_1 , suggesting error feedback. Red solid arrows connect the predicted outputs and ground truth to the subtraction node, which outputs the 'Classifier Error'. This error signal is visually emphasized with bold red lines and text, indicating its role in training or refinement.



The figure illustrates the main components of a method for feature extraction and data imputation in a sports video analysis context, structured into three panels labeled (a), (b), and (c). Panel (a) depicts the feature extraction pipeline. On the left, an input image of a soccer game is processed through two parallel pathways. The top pathway feeds the entire image into a CNN (represented as a trapezoid) to extract 'whole-image feat.' (whole-image features), shown as a blue vertical bar. The bottom pathway first applies a 'Detection' module (a rectangle) to identify and localize players in the image, producing a set of bounding boxes around individuals. These detected regions are then fed into another CNN to extract 'foreground feat.' (foreground features), represented by an orange vertical bar. The two feature vectors are then combined vertically to form a combined feature vector. For the auxiliary game, the extracted features undergo an additional 'Imputation' step (rectangle) to handle missing data. The imputed features are also represented as multi-colored bars. A red curved arrow connects the imputed auxiliary features back to the main feature vector from panel (a), indicating that these imputed features are integrated into the overall training process. This suggests that the auxiliary game data, after imputation, contributes to training the final model alongside the main game data. Panel (b) outlines the training process. It begins with two types of game footage: 'Main game' and 'Aux. game' (auxiliary game), each shown as a video frame. Features are extracted from both using an 'Extraction' module (rectangle), producing multi-colored horizontal bars representing feature sequences. For the auxiliary game, the extracted features undergo an additional 'Imputation' step (rectangle) to handle missing data. The imputed features are also represented as multi-colored bars. A red curved arrow connects the imputed auxiliary features back to the main feature vector from panel (a), indicating that these imputed features are integrated into the overall training process. This suggests that the auxiliary game data, after imputation, contributes to training the final model alongside the main game data. Panel (c) details the data imputation mechanism, specifically referencing a Random Survival Forest approach. At the top, a legend defines the data types: black circles for 'Complete' data points, white circles for 'Incomplete' data points, and gray circles for 'Imputed' data points. Below, a tree structure is shown with blue circular nodes representing decision or internal nodes. The tree branches down to leaf nodes. One leaf node is highlighted with a dashed oval, showing an incomplete data point (white circle) being transformed into an imputed data point (gray circle), illustrating how missing values are estimated based on the learned tree structure. This visualizes the imputation process where complete observations are replaced with predicted values derived from the model's learned patterns. Overall, the figure presents a complete workflow: (a) extracting comprehensive features from game images via dual CNN pathways; (b) training a model using both main and auxiliary game data, with auxiliary data undergoing imputation to handle incompleteness; and (c) detailing the imputation technique using a tree-based model that predicts missing values based on observed data patterns.

